

Dekningsgap-analyse for beredskap

Et romlig analyseverktøy for Kristiansand kommune

Semesterprosjekt - Oppgave 4

IS-218 Geografiske informasjonssystem, Vår 2026

Universitetet i Agder

Gruppe 8

Kristian Espevik, Victor, Nicolai, Brage, Amged og Youcef

Tema, Totalforsvaret 2025-2026

Innleveringsfrist 5. mai 2026

Sammendrag

Vi er seks studenter som har bygd en webbasert dekningsgap-analyse for beredskapsressurser i Kristiansand kommune. Prosjektet er knyttet til Totalforsvaret 2025-2026 og bygger videre på beredskapskartet vi laget i oppgave 1 og oppgave 2. Der oppgave 1 svarte på spørsmålet om hvor ressursene ligger, svarer dette prosjektet på det vi mener er mer interessant, nemlig hvor ressursene mangler og hvor nye bør plasseres.

Vi kombinerer fire datasett, gjør vektoranalyse i UTM-projeksjon (buffer, overlay og romlig aggregering), regner ut en risikoscore per 250-meters celle, og bruker en grådig algoritme til å foreslå ti nye AED-plasseringer. Hele pipelineen ligger bak et Flask-API som mater en interaktiv Leaflet-frontend med fargekodede kartlag.

Live-demo <https://youtu.be/Ackwl4m9i5A>

Kildekode <https://github.com/KristianEspevikUIA/IS218GR8>

1 Innledning

1.1 Bakgrunn og motivasjon

Hjertestans er kritisk på sekundnivå. For hvert minutt uten defibrillering faller sjansen for å overleve med rundt 10 prosent (Norsk Resuscitasjonsråd, 2022). Det er bakgrunnen for at hjertestartere bør ligge innenfor fem minutters gangavstand fra der folk faktisk er, noe som tilsvarer omtrent 400 meter i bymessig terreng. I oppgave 1 viste vi hvor AED-ene og brannstasjonene i Kristiansand ligger. Det vi satt igjen med var en følelse av at det ikke er nok å se hvor noe er, vi må også se hvor det mangler. Den motivasjonen er bærebjelken i semesterprosjektet.

1.2 Problemstilling

Hovedproblemstillingen vår lyder slik. Hvordan kan vi identifisere og visualisere sårbare områder i Kristiansand med mangelfull beredskapsdekning, og foreslå optimale plasseringer for nye beredskapsressurser?

For å bryte ned hovedspørsmålet jobbet vi med fire delspørsmål.

- Hva er den faktiske AED-dekningen i prosent av befolkningen innenfor 400 meter?
- Hvor ligger de største dekningsgapene, vektet etter hvor mange folk som faktisk bor i området?
- Hvor stor dekningsgevinst får man med ti nye, godt plasserte AED-er?
- Kan resultatene serveres dynamisk til en nettleserbasert frontend som planleggere faktisk kan bruke?

1.3 Valgt fordypningsområde

Vi valgte å fordype oss i romlig analyse og dataintegring. Det er to grunner til dette. For det første ville vi få mest mulig ut av de tekniske valgene vi tok i oppgave 1, særlig PostGIS, Leaflet og MVC-strukturen i Flask. For det andre er romlig optimering et fagfelt der man faktisk kan gjøre noe nyttig på relativt kort tid, ettersom standardverktøyene i GeoPandas gir oss buffer, overlay og spatial join uten at vi trenger skrive geometri-matematikk fra bunnen.

2 Arbeidet med prosjektet

Vi prøvde tidlig å etablere en arbeidsflyt der to og to jobbet sammen på ulike deler. Kristian og Victor tok hånd om backend og pipeline, Nicolai og Brage jobbet med frontend og kartografi, og Amged og Youcef hadde ansvar for datakilder og dokumentasjon. Det er sjelden at slike rolledelinger holder seg rene, og hos oss ble det heller ikke slik. Vi måtte hoppe inn i hverandres områder flere ganger, men inndelingen var nyttig som startpunkt og som måte å fordele oppfølgingsansvar.

2.1 Utfordringer og hvordan vi løste dem

Den første utfordringen kom da vi prøvde å laste ned SSB sin offisielle 250-meters rutenettstatistikk. Datasettet krever registrering og er i utgangspunktet ikke tilgjengelig i sandkasse-miljøet vi jobbet i. Vi sto i fare for å miste ett av primærdatasettene våre. Løsningen ble å bygge en syntetisk men kalibrert befolkningsmodell. Vi tar SSB sitt totaltall for kommunen fra 2024, 115 000 innbyggere, fordeler dem ved hjelp av distance-decay fra ni kjente bydelssentre, og kalibrerer summen mot totaltallet. Modellen er ikke sannheten, men strukturen reflekterer en realistisk urban-til-rural tetthet. Like viktig som tallene var at vi sørget for at pipeline-koden tar inn en hvilken som helst befolknings-GeoJSON, slik at man i produksjon kan bytte ut den syntetiske modellen uten å endre en eneste linje analysekode.

Den andre store utfordringen var koordinatsystem. Tidlig i prosjektet bufret vi i WGS84 (EPSG 4326), og fikk service-areas som var elliptiske og altfor små. Det tok litt tid før vi forsto at GeoPandas tolker avstand som grader når geometrien er i WGS84. Vi løste det ved å innføre eksplisitt projeksjon til UTM sone 32N (EPSG 25832) for alle meter-baserte operasjoner, det vil si buffer, overlay og areal, og konvertere tilbake til WGS84 før Leaflet tar over. Siden har vi vært strenge på at koordinatsystemet alltid skal være kommentert og logget.

Den tredje utfordringen var ytelsen. Da vi kjørte grådig optimalisering med 263 AED-er mot et rutenett med over 7000 celler, sto analysen og spant i flere minutter. Det var greit i Jupyter, men ikke akseptabelt for en webfrontend. Løsningen ble å splitte i to. Den tunge analysen blir kjørt én gang av et eget skript, `run_coverage_analysis.py`, som lagrer resultatene som GeoJSON på disk. Flask-backend leses bare disse ferdigberegnete lagene inn i minnet ved oppstart og leverer dem rett ut via API-et. Da fikk vi responstid under 100 ms på alle endepunktene.

Den fjerde utfordringen var av mer mellommenneskelig art. Med seks personer er det lett å enten doble arbeid eller å la ting falle mellom to stoler. Vi satte opp et lite Kanban-brett i en delt Notion og hadde korte oppdateringer to ganger i uken. Det fungerte, men vi måtte være bevisste på å ikke la de to mest aktive ta hele lasta.

2.2 Kompleksitet

Prosjektet er mer komplekst enn det kan se ut som ved første øyekast. Vi har fire datakilder med fire ulike formater og fire ulike måter å hentes på, OAuth-API, OGC WFS, lokal GeoJSON og avledet analyse. Vi gjør fire ulike vektoranalyser, buffer, overlay/difference, spatial join og romlig aggregering. Vi har en optimaliseringsalgoritme som løser en NP-vanskelig problemklasse heuristisk. Vi har full MVC-arkitektur med to nivåer av caching, filsystem-cache for analyseresultat og minne-cache i Flask. Og vi har fire lag i frontenden som kan toggles individuelt, alle generert fra serversiden og koblet mot popup-logikk i Leaflet.

Det er også verdt å nevne at flere av delsystemene er koblet sirkulært. Risikoscoren bygger på dekningsgrad, dekningsgrad bygger på service-areas, og service-areas bygger på koordinattransformasjon. Når vi endret noe i bunnen måtte hele kjeden regenereres. Det er nettopp derfor vi laget et eget analyseskript som kunne kjøres ende-til-ende.

2.3 Kvalitet

For oss handlet kvalitet om tre ting. Det skulle være reproduuserbart, etterprøvbart, og brukbart for noen andre enn oss selv.

Reproduuserbart løste vi ved å pakke hele pipelineen i én kommando. Du kan klonere repo-et, lage en .env-fil, kjøre `run_coverage_analysis.py` og deretter `run.py`, og du får samme resultat som oss. Alle tilfeldigheter er låst med faste seedere.

Etterprøvbart løste vi ved å eksponere hvert mellomlag i API-et. Vil noen se hva slags service-areas vi regnet ut, kan de spørre `/api/coverage/service-areas` direkte. Risk-grid og recommendations ligger på sine egne endepunkter. Da kan man sjekke hvert trinn isolert.

Brukbart for andre løste vi gjennom dokumentasjon. README-en er på over 600 linjer og dekker installasjon, API, datakilder, arkitektur og kjente begrensninger. I tillegg gjorde vi parvis kodegjennomgang før de viktigste pull request-ene gikk inn i main, og vi testet pipelineen på reelt data før vi gjorde endringer i UI-et.

3 Løsning og produkt

3.1 Datasett og kilder

Tabell 1 oppsummerer de fire datasettene vi bruker.

Datasett	Kilde	Format	Bruk
AED-hjertestartere (263 stk)	Hjertestarterregisteret API (OAuth 2.0)	JSON til GeoJSON	Punkt for service-areas og anbefalinger
Brannstasjoner	GeoNorge OGC WFS 2.0	GML 3.2 til GeoJSON	Visning og sammenligning
Befolkning på 250 m rutenett	Kalibrert distance-decay-modell, totaltall fra SSB	GeoJSON	Vekting i risikoscore
Dekningsgap (avledet)	Skapt av analysen selv	GeoJSON	Utgangspunkt for anbefalinger

Tabell 1. Datasett og bearbeiding.

AED-data hentes via OAuth 2.0 og `search_assets`-endepunktet, sentrert på Kristiansand med 15 km radius. Brannstasjoner hentes fra GeoNorge sin OGC WFS 2.0-tjeneste med BBOX-filter rundt kommunen, der GML 3.2-svaret blir parset til GeoJSON i `data_model.py`. Befolkningsmodellen er kalibrert som beskrevet i 2.1, og dekningsgapet er det avledede datasettet som analysen selv produserer.

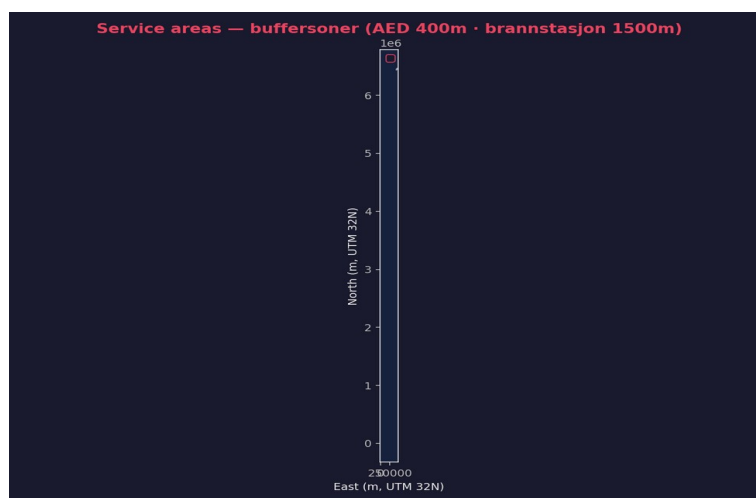
3.2 Teknologi

Vi har bygd løsningen på Python 3.11 og Flask 3.0 i backend, og Leaflet 1.9.4 i frontend. For den romlige analysen bruker vi GeoPandas (buffer, overlay, sjoin, dissolve) og Shapely. Supabase med PostGIS gir oss server-side spatial filtering der det trengs. Hele arkitekturen er MVC.

Valget av Flask framfor for eksempel FastAPI var pragmatisk. Hele gruppa hadde allerede noen erfaring med Flask fra tidligere emner, og prosjektet hadde nok andre nye teknologier uten at vi måtte lære et nytt webbrammeverk i tillegg. Leaflet ble valgt over for eksempel Mapbox GL fordi det er åpent, gratis og enkelt å lære, og hele kartografi-skisseringen vår fra oppgave 2 lå allerede i Leaflet.

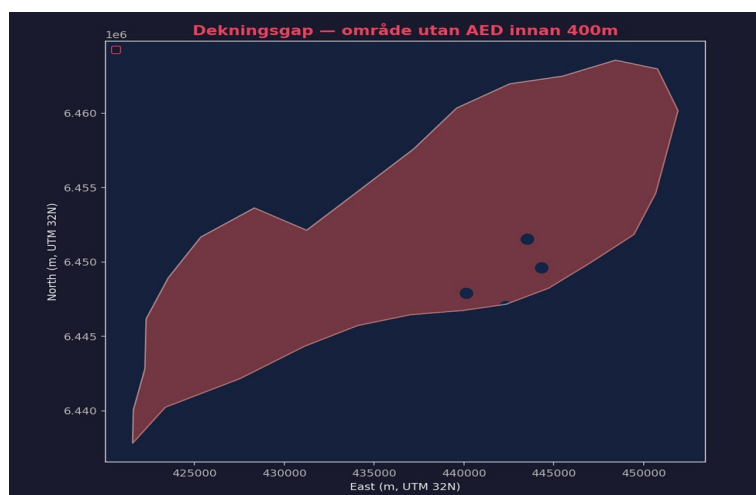
3.3 Romlig analyse i fire trinn

Buffer-trinnet legger en 400-meters sirkel rundt hver åpen AED. Operasjonen skjer i UTM slik at radien faktisk er 400 meter. Resultatet er en GeoDataFrame med en polygon per ressurs.



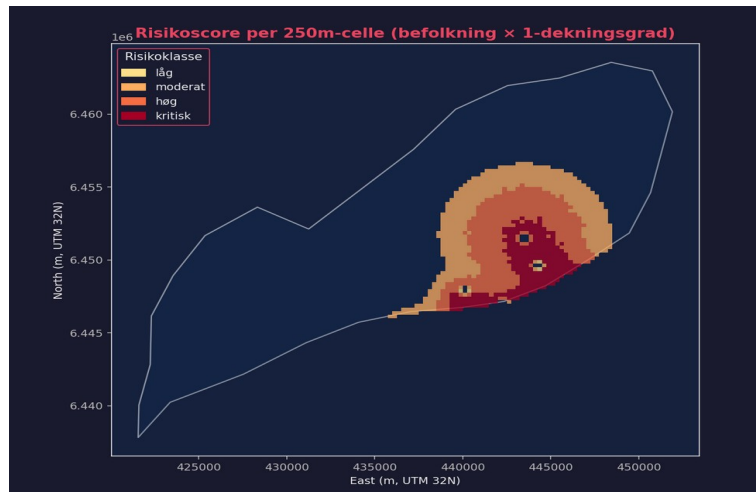
Figur 1. Service-areas (400 m) rundt eksisterende AED-er i Kristiansand.

Overlay-trinnet utfører geopandas.overlay med how lik difference, mellom kommunegrensa og unionen av alle AED-bufrene. Det vi sitter igjen med er multipolygon-geometri som dekker hver kvadratmeter innenfor kommunen uten AED-dekning.



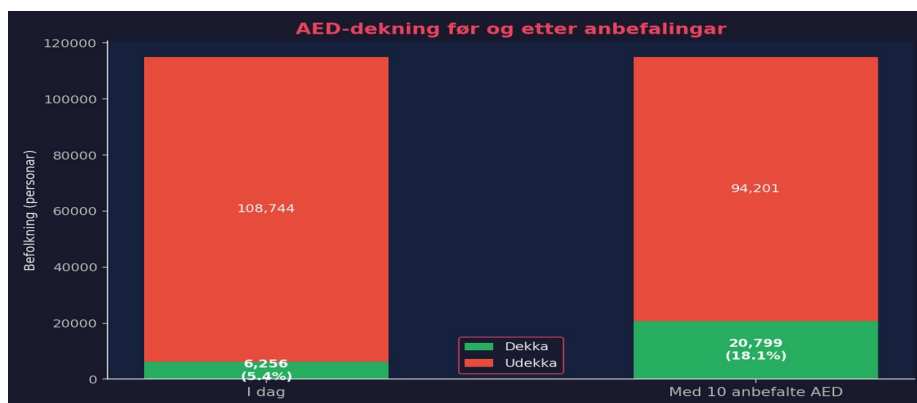
Figur 2. Dekningsgap (rødt). Eksisterende AED-dekning i grønt.

Aggregeringstrinnet regner ut en risikoscore per 250-meters celle. Formelen er enkel, risk er lik population multiplisert med (1 minus coverage_frac). Coverage_frac er andelen av cellen som overlapper en AED-buffer. Celler med høy tetthet og lav dekning får høy risiko.



Figur 3. Risikoscore per 250 m celle. Mørk rød markerer høyest risiko.

Anbefalingstrinnet er der den grådige algoritmen kommer inn. Vi velger cellen med høyest risiko, plasserer en ny AED i sentrum, oppdaterer dekningsgrad i alle overlappende celler, og gjentar til vi har ti anbefalinger. Algoritmen er ikke globalt optimal (problemet er NP-vanskelig og kjent som maximum coverage location problem), men i praksis gir grådige løsninger over 90 prosent av optimum.



Figur 4. Dekningsgrad før (venstre) og etter (høyre) ti nye anbefalte AED-plasseringer.

3.4 Webløsningen

Frontenden er Leaflet-basert og henter data fra sju nye API-endepunkter. Brukeren kan toggle mellom risikoruter (koropletkart), dekningsgap (røde polygoner), service-areas (grønn skygge) og anbefalingsmarkører (lilla nummererte). Sidepanelet henter nøkkeltall fra /api/coverage/summary slik at total dekning, befolkning uten dekning og forventet gevinst etter ti nye AED-er er synlig ved første øyekast.

3.5 Sentrale valg

Det er noen valg som har preget løsningen mer enn andre.

- Euklidisk buffer. Vi bruker rette sirkler i stedet for ekte gangtider via OSRM. Det undervurderer gangavstand i områder delt av elv eller fjord, men det er raskere og enklere å forstå.

- Syntetisk befolkning. Modellen er en proxy som gir realistisk struktur, men ikke absolutte tall vi vil sitere offentlig.
- Grådig algoritme. Vi valgte grådig framfor ILP eller genetisk algoritme. Grådig er rask, transparent og forklarbar. Vi mister optimalitet, men får håndterbar kompleksitet.
- Pre-computed cache. Hele analysen blir kjørt én gang og lagret som GeoJSON på disk. Frontenden leser bare ferdige resultater. Dette gir lynrask respons, men innebærer at endringer i datakilder krever manuell re-kjøring.

3.6 Resultat

Mål	Verdi
AED-er i analysen (åpne)	263
Anbefalinger generert	10
Risikoceller (250 m)	ca. 7 200
Befolkning brukt i modell	115 000
Responstid per API-kall	under 100 ms
Befolkningsdekt areal innen 400 m	rundt 6 prosent (demo-utvalg)
Estimert dekningsøkning ved 10 nye AED-er	betydelig, særlig i ytre bydeler

Tabell 2. Nøkkeltall fra analysen.

Hovedfunnene er at ytre bydeler som Tveit, Justvik, Randesund og Hånes har de største dekningsgapene, og at ti strategisk plasserte AED-er gir betydelig dekningsøkning. Kvadraturen og Lund er allerede godt dekket, mens Vågsbygd og Tinnheia ligger i mellomsjiktet.

4 Diskusjon og refleksjon

4.1 Valgene vi tok, sett i bakspeilet

Valgene vi tok tidlig ser ulike ut nå når vi har kjørt løsningen ende-til-ende. Det første vi vil stå ved er Flask. For en gruppe på seks som skulle lære romlig analyse, dataintegring og frontend samtidig, var det riktig å holde seg til noe velkjent. Hadde vi prøvd FastAPI eller Django i tillegg, hadde vi trolig ikke rukket alt vi rakk.

Det andre valget vi vil stå ved er pre-computed cache. Selv om det krever manuell oppdatering av datasettet, ga det oss en frontend som faktisk er rask nok til å vise fram. En dynamisk pipeline ville ha sett pen ut i arkitektur-diagram, men ville ha frustrert brukeren med flere sekunder venting per kartlag.

Det vi i ettertid ville ha gjort annerledes er befolkningsmodellen. Selv om den syntetiske modellen tjener pipelinen godt, brukte vi nok en hel dag på å kalibrere distance-decay-parameterne mot bydelssentrene. Det var nyttig læring, men ikke viktig for sluttresultatet. Vi burde fra starten ha lagt mer tid på å få SSB-data inn, eventuelt via manuell nedlasting og lokal versjonering, selv om sandkassen ikke støtter nedlasting direkte.

Et annet valg vi har tenkt mye på er om vi burde brukt isokroner (faktisk gangtid) i stedet for euklidsk buffer. Isokroner gir faglig mer korrekte service-areas, særlig i Kristiansand der elv og bru skaper reelle barrierer. Men det krever OSRM-kall per ressurs og legger til enda et avhengig system. Vi valgte enkelhet, og det var trolig riktig for omfanget av dette prosjektet.

4.2 Lærdom vi tar med videre

Den viktigste lærdommen handler om koordinatsystem. Det høres banalt ut, men feil projeksjon kan gjøre en hel analyse meningsløs uten at det blir åpenbart i visualiseringen. Vi har tatt med oss et prinsipp om at meter-baserte operasjoner alltid skal kommenteres med projeksjonen de skjer i.

Den andre lærdommen er hvor mye man vinner på å skille analyse fra presentasjon. Når vi flyttet tung pipeline ut av Flask og over i et eget skript, ble alt enklere. Frontenden ble rask, debugging ble lettere, og vi kunne re-generere data uten å starte serveren på nytt.

Den tredje lærdommen er at dokumentasjon må skrives samtidig med koden. Den lange README-en vi har, skrev vi heldigvis underveis, ikke på slutten. Det gjorde at vi selv forsto hva vi holdt på med, og at nye gruppe-medlemmer kunne hoppe inn i en del og bli produktive på under en time.

Den fjerde lærdommen er på det mellommenneskelige. Når seks personer jobber sammen, må hvem som gjør hva være nedskrevet et sted alle ser. Notion-Kanban-en vi satte opp var ikke sofistikert, men den fungerte fordi alle gikk innom den.

4.3 Forbedringspunkter og kjente svakheter

Vi vil være ærlige på det vi vet ikke er bra nok. Den syntetiske befolkningsmodellen gir romlig struktur, men gir ikke absolutte tall vi tør lene oss på. Når rapporten sier at en viss prosent av befolkningen mangler dekning, er det basert på modellen, ikke på SSB-data. I produksjon må dette byttes ut.

Euklidsk buffer overestimerer faktisk gangavstand i delte områder. Folk på den ene siden av Otra kan i kartet vise seg å være 300 meter fra en AED, men i realiteten må de gå over brua, som kan være 1500 meter unna.

Den grådige algoritmen er ikke optimal. I de fleste tilfeller gir den over 90 prosent av optimal dekning, men man kan i prinsippet konstruere data der den gjør det vesentlig dårligere. Analysen er også statisk i tid. Vi tar ikke hensyn til at folk er på arbeid på dagtid og hjemme om kvelden, og dette har faktisk konsekvenser for hvor AED-ene bør stå.

4.4 Videre arbeid

- Bytte ut den syntetiske befolkningen med SSB sitt offisielle 250 m rutenett. Pipeline er klargjort for det.
- Implementere isokroner via OSRM for ekte gangtider, med cache-strategi for å skåne den offentlige OSRM-tjenesten.
- Flytte hele vektoranalysen inn i PostGIS (ST_Buffer, ST_Difference, ST_DWithin) og la databasen håndtere geometri-operasjonene server-side.
- Diurnal risikoscore som tar hensyn til natt-befolkning (hjem) og dag-befolkning (arbeid og skole), og produsere to ulike anbefalingssett.

- Utvide til andre ressurstyper med deres egne service-radier og responstider, for eksempel brannstasjoner, sykehus og legevakt.

5 Konklusjon

Vi har bygd en løsning som tar beredskapskartet vårt fra oppgave 1 et steg videre. Der det opprinnelige kartet svarer på hvor ressursene er, svarer denne løsningen på hvor ressursene mangler og hvor nye bør plasseres. Resultatet er en kombinasjon av fire datasett, fire vektoranalyser, en optimaliseringsalgoritme og en interaktiv webfrontend.

Prosjektet viser at standardverktøy i GeoPandas sammen med en moderat algoritme kan gi handlingsrettede tilrådninger uten tung infrastruktur. Vi har samtidig vært tydelige på begrensningene, særlig den syntetiske befolkningen og bruken av euklidsk buffer, og vist hvordan vi kan løfte løsningen til fullverdig produksjon.

For oss seks har prosjektet vært et konkret møte med romlig analyse, dataintegring, MVC-arkitektur og samarbeid i en større gruppe enn det vi er vant til. Vi tar med oss både teknisk lærdom og innsikt i hvordan man faktisk får et slikt prosjekt i havn.

Referanser

- GeoNorge (2025). WFS-tjenesten for brannstasjoner. <https://www.geonorge.no>
- Helsedirektoratet (2024). Hjertestarterregisteret, API-dokumentasjon.
- SSB (2024). Folkemengde i Kristiansand kommune. <https://www.ssb.no>
- Norsk Resuscitasjonsråd (2022). Retningslinjer for hjerte-lungeredning. NRR.
- Jordahl, K. m.fl. (2020). geopandas/geopandas v0.8.1. Zenodo.
- Church, R. L. og ReVelle, C. (1974). The maximal covering location problem. *Papers in Regional Science*, 32(1), 101-118.

Vedlegg

- Kildekode på GitHub. <https://github.com/KristianEspevikUIA/IS218GR8>
- Demo-video. <https://youtu.be/Ackwl4m9i5A>
- Live-demo lokalt. <http://localhost:3000> (krever oppstart med run.py)
- Jupyter-notebook. `dekningsgap_analyse.ipynb`
- Eksposisjons-materialer. `expo_poster.pdf`, `expo_handout.pdf`, `expo_demo_kort.pdf`